

TECHNICAL SPECIFICATION

Minecraft Server Checker

Windows向け単一EXE GUIツール 詳細仕様書

項目	内容
文書版	1.1
作成日	2026-08-18
対象	個人利用 / Windows 10・11 / Minecraft Java Edition
推奨技術	Go + Fyne v2 / Minecraft Status Protocol 自前実装
MVP成果物	mc-server-checker.exe（単一ファイル）

採用判断 MVPはFyneを採用する。WebViewランタイムを要求せず、配布物を単一EXEに保ちやすい。通信層はGUIから分離し、将来のBedrock・監視・JSON出力へ拡張できる構成とする。

文書の目的

本書は、アドレスとポートを入力してMinecraft Java Editionサーバーの稼働状況を確認する、個人用Windows GUIツールの実装基準を定める。設計判断、画面、通信、エラー、ビルド、テスト、将来拡張までを一貫した受入条件として扱う。

スコープ要約

- MVP: Java Editionのオンライン/オフライン、Ping、バージョン、プレイヤー数、MOTDを表示。
- 配布: Windows用の単一EXE。インストーラー、管理者権限、常駐サービスは不要。
- 対象外: サーバー起動・停止、RCON、ログイン、プレイヤー操作、ポート開放診断。
- 拡張余地: Bedrock、自動判定、定期監視、履歴、通知、JSON出力。

目次

章	内容
1	目的・前提・用語
2	技術選定と採用判断
3	画面・操作仕様
4	機能要件
5	非機能要件
6	エラー処理
7	内部構成・パッケージ構成
8	データモデルとデータフロー
9	Minecraft Status Protocol
10	ビルド・配布
11	テスト仕様
12	開発ステップ
13	将来拡張
14	MVP受入基準
付録	既定値・参考情報・変更方針

読み方 実装者は2章の採用判断、4章の要件ID、9章のプロトコル制約、11章のテストをセットで参照する。要件変更時は関連する受入条件も同時に更新する。

1. 目的・前提・用語

1.1 利用シナリオ

利用者はEXEを起動し、サーバーのホスト名またはIPアドレスとポートを入力して「確認」を押す。アプリはMinecraft Java EditionのServer List

Pingを実行し、結果を同一画面に表示する。確認は1回単位で、バックグラウンド常駐しない。

1.2 前提

- Windows 10またはWindows 11、x64を初期対象とする。
- インターネットまたは対象LANへの到達性があること。
- 対象はJava EditionのStatus要求に応答するサーバーまたは互換プロキシであること。
- サーバー側設定、ファイアウォール、DNSは本ツールの管理対象外とする。

1.3 用語

用語	定義
オンライン	TCP接続後、Status Responseを制限時間内に正しく解釈できた状態。
オフライン	接続拒否、タイムアウト、DNS失敗等で有効なStatus Responseを得られない状態。
Ping	Ping Request送信から対応するPong受信までの往復時間。ミリ秒表示。
MOTD	Status Responseのdescriptionから抽出したプレーンテキスト。装飾はMVPでは簡略化。
単一EXE	利用時に同梱DLL、Node.js、Java、WebView固定版、設定ファイルを必須としない実行形式。

2. 技術選定と採用判断

2.1 GUI候補比較

候補	長所	短所・注意	判断
Fyne v2	Go中心。Windows用EXEへリソース埋込可能。実行時の外部UIランタイム不要。	ビルドにCコンパイラ/CGO。外観はFyne独自。	採用
Wails v2	HTML/CSSで高い表現力。Goとのバインディングが明快。単一バイナリ生成。	Windows実行時にWebView2が必要。フロントエンド/Node系ビルドが増える。	次点
Walk	Win32ネイティブ外観。Goから直接構築。軽量。	Windows専用。manifest埋込が必須。保守性・将来性のリスクが相対的に高い。	不採用

評価軸: 単一EXE適合、個人開発の簡潔さ、実行時依存、保守性、将来拡張。

2.2 採用案

推奨 Go + Fyne v2 + 標準netパッケージ + encoding/json。Minecraft Status Protocolは小さな独立パッケージとして自前実装し、GUI依存を持たせない。

自前実装の対象はStatusに必要な最小範囲のみとする。ログインやPlay状態の完全実装は行わない。外部Minecraftライブラリを追加する場合も、protocol.Checkerインターフェースの背後に隔離し、UIとテストを変更せず差し替え可能にする。

2.3 バージョン方針

項目	方針
Go	開発開始時点の安定版を固定し、go.modのgo/toolchainで再現性を確保。
Fyne	v2系の特定バージョンをgo.modで固定。無条件のlatest依存を避ける。
Windows	MVPはwindows/amd64。arm64はCIと実機確認後に追加。
更新	依存更新は別ブランチでビルド・起動・UI・通信回帰テストを実施。

3. 画面・操作仕様

3.1 メイン画面

Minecraft Server Checker

×

Address [play.example.com]
Port [25565] [確認]

● ONLINE 最終確認 2026-08-18 21:05:13

Ping 24 ms
Version Paper 1.21.x
Players 4 / 20
MOTD Example Minecraft Server

[結果をコピー] v1.0.0

3.2 画面要素

ID	要素	仕様
UI-01	Address	必須。ホスト名、IPv4、角括弧付きIPv6を入力。前後空白は除去。
UI-02	Port	1～65535。初期値25565。数値のみ。
UI-03	確認	入力妥当時のみ実行。処理中は無効化し、ラベルを「確認中…」へ変更。
UI-04	状態	未確認 / 確認中 / ONLINE / OFFLINE / ERRORを色と文字で表示。
UI-05	結果	Ping、Version、Players、MOTD。値なしは「—」。
UI-06	最終確認	ローカル時刻。結果が更新された時のみ変更。
UI-07	結果をコピー	結果を読みやすい複数行テキストとしてクリップボードへコピー。

3.3 状態遷移

現在	イベント	次状態	表示・操作
未確認	確認押下	確認中	旧結果を薄く保持、ボタン無効、進捗表示。
確認中	成功	ONLINE	結果を更新し緑表示。
確認中	到達失敗	OFFLINE	理由を短文化し赤表示。
確認中	応答不正	ERROR	不正応答として黄/赤表示。
確認中	再押下	確認中	無視。二重要求を作らない。
任意	入力変更	未確認	既存結果は残すが「前回結果」と明示して誤認を防ぐ。

3.4 操作性

- Enterキーで確認を開始する。Tab順はAddress → Port → 確認 → コピー。
- 結果は色だけで判別させず、ONLINE/OFFLINEの文字と状態アイコンを併用する。
- ウィンドウ初期サイズは約560×430 px、最小サイズは520×380 px。結果欄はMOTDに応じて折返す。
- OSの表示倍率100～200%で文字やボタンを切らさない。ライト/ダークはFyneのシステムテーマ追従を基本とする。

4. 機能要件

ID	名称	要件	優先度
FR-01	接続先入力	AddressとPortを受け付ける。Port省略時は25565。	必須
FR-02	入力検証	空欄、範囲外Port、不正な角括弧IPv6を通信前に拒否。	必須
FR-03	DNS/TCP	名前解決後、TCPで接続。contextによる全体期限を適用。	必須
FR-04	Status取得	Handshake → Status Request → Status Responseを実行。	必須
FR-05	Ping計測	Ping/Pongの往復時間を1 ms単位で表示。	必須
FR-06	基本結果	ONLINE/OFFLINE、Version、Players、MOTDを表示。	必須
FR-07	再実行	同一/別接続先へ連続して確認可能。リソースを都度解放。	必須
FR-08	非同期UI	ネットワーク処理中でもウィンドウ操作を固めない。	必須
FR-09	コピー	接続先、状態、時刻、主要結果をクリップボードへコピー。	推奨
FR-10	設定保持	最後に成功したAddress/PortをFyne Preferencesに保存。秘密情報は保存しない。	推奨
FR-11	キャンセル	ウィンドウ終了時に実行中contextをcancelし、接続を閉じる。	必須
FR-12	ログ	通常はファイル出力しない。開発ビルドのみ標準エラーへ診断情報。	推奨

4.1 結果表示規則

フィールド	表示規則
Target	入力を正規化したhost:port。IPv6は[addr]:port。
Status	ONLINE / OFFLINE / ERROR。未確認値と混同しない。
Ping	math.Round相当で整数ms。計測不可は「—」。0未満を許容しない。
Version	version.name。空なら「不明」。protocol番号は詳細情報として内部保持。
Players	online / max。片方欠落時は取得できた値だけ表示。負数は不正応答。
MOTD	descriptionをプレーンテキスト化。改行は最大2行、制御文字除去、長文はUIで折返し。

5. 非機能要件

ID	分類	基準
NFR-01	応答性	ローカル/正常回線では通常3秒以内。全体タイムアウト既定5秒、最大30秒。
NFR-02	UI	ネットワーク処理をgoroutineで実行し、UI更新はFyneのメインスレッド規約に従う。
NFR-03	配布	利用者向け配布物はEXE 1個。管理者権限・インストール不要。
NFR-04	互換性	Windows 10/11 x64、表示倍率100/125/150/200%。
NFR-05	資源	単発確認後にTCP接続・timer・goroutineが残留しない。
NFR-06	安全性	受信パケット上限1 MiB、MOTD表示上限4 KiB相当。無制限確保を禁止。
NFR-07	保守性	GUI、アプリケーション、プロトコル、永続化をパッケージ分離。protocolはUIなしで単体テスト可能。
NFR-08	再現性	依存バージョン固定。リリースビルド手順とハッシュを記録。
NFR-09	プライバシー	テレメトリ、広告、外部分析、クラウド送信を行わない。
NFR-10	アクセシビリティ	状態を色のみで示さない。キーボード操作と高DPIを確認。

6. エラー処理

内部エラーは分類可能な型として保持し、画面では短い日本語メッセージと再試行ヒントに変換する。OSの生エラー全文やスタックトレースは通常画面に出さない。

分類	代表条件	画面表示例	状態
Validation	Address空、Port範囲外	入力内容を確認してください。	ERROR
DNS	名前解決失敗	ホスト名を解決できません。	OFFLINE
Timeout	接続/読取期限超過	時間内に応答がありませんでした。	OFFLINE
Refused	接続拒否	接続が拒否されました。ポートを確認してください。	OFFLINE
Network	経路なし、切断	ネットワークに接続できませんでした。	OFFLINE
Protocol	VarInt不正、packet ID違い	Minecraft Status応答を解釈できません。	ERROR
Payload	JSON不正、上限超過	サーバー応答が不正または大きすぎます。	ERROR
Cancelled	終了/将来のキャンセル	確認を中止しました。	未確認/前回結果

6.1 判定原則

- TCPポートが開いていてもMinecraft Status Responseを得られなければONLINEとしない。
- Status Response取得後にPing/Pongだけ失敗した場合はONLINEを維持し、Pingを「取得失敗」とする余地を残す。MVP実装では警告付きONLINEとする。
- 未知のJSONフィールドは無視し、必須表示フィールドの欠落は「不明」として可能な範囲で成功扱いにする。
- panicは境界で回復して通常エラーへ変換する。ただしプログラミング不具合を隠さないよう開発ビルドに診断を残す。

7. 内部構成・パッケージ構成

設計原則 依存方向は ui → app → protocol / storage. protocolとdomainはFyneをimportしない。これにより将来CLI/JSON出力を同じ通信層で追加できる。

7.1 論理コンポーネント

層	責務	主な型
ui	入力、状態表示、イベント、クリップボード、テーマ	MainWindow, ViewModel
app	ユースケース実行、入力正規化、キャンセル、エラー文言	CheckService
domain	結果・状態・エラー分類。外部依存なし	Target, Result, AppError
protocol/java	TCP、packet codec、Status/Ping、JSON変換	Client, Codec, StatusPayload
storage	最後の接続先等の非機密設定	Preferences
platform	ビルド情報、アイコン、Windows固有設定	BuildInfo

7.2 推奨ディレクトリ

```
mc-server-checker/
cmd/mc-server-checker/main.go
internal/app/check_service.go
internal/domain/{target,result,error}.go
internal/protocol/java/{client,codec,status,motd}.go
internal/storage/preferences.go
internal/ui/{main_window,view_model,theme}.go
assets/{icon.png,icon.ico}
build/windows/{app.manifest,versioninfo.json}
go.mod go.sum FyneApp.toml
Makefile.ps1 README.md
```

7.3 主要インターフェース

```
type Checker interface {
    Check(ctx context.Context, target domain.Target) (domain.Result, error)
}

type CheckService struct { checker Checker; timeout time.Duration }
```

8. データモデルとデータフロー

8.1 ドメインモデル

型	フィールド	制約
Target	Host string; Port uint16	Hostは表示用と接続用を分けず、net.JoinHostPortで整形。
Result	Target, Status, Latency, VersionName, Protocol, PlayersOnline, PlayersMax, MOTD, CheckedAt, Warning	Status Response成功時に生成。欠落値はポインタ/okで区別。
Status	Unknown, Checking, Online, Offline, Error	UI状態と通信結果を明確に区別。
AppError	Kind, Message, Cause, Retryable	Causeは画面へ直接表示しない。errors.Is/As対応。
StatusPayload	Version, Players, Description, Favicon, EnforcesSecureChat等	未知フィールド許容。UIへ直接渡さずResultへ変換。

8.2 単発確認データフロー

1. UIがAddress/Portを読み、CheckServiceへ渡す。
2. CheckServiceがtrim、Port検証、Target生成を行う。
3. 全体タイムアウト付きcontextを生成し、Checker.Checkをgoroutineで実行する。
4. Java ClientがDNS/TCP接続、Handshake、Status Request、Status Responseを処理する。
5. Status JSONを制限付きでdecodeし、MOTDをプレーンテキスト化する。
6. Ping Request/Pongを実行しRTTを計測する。
7. ResultまたはAppErrorをUIスレッドへ返し、状態・表示・コピー内容を一括更新する。
8. deferで接続を閉じ、context、timer、goroutineを回収する。

8.3 同時実行制御

MVPは同時に1要求のみ許可する。確認中はボタンを無効化し、requestIDを採番する。将来キャンセルや監視を追加した場合、完了通知のrequestIDが現在値と一致しない古い結果はUIへ反映しない。

9. Minecraft Status Protocolの扱い

範囲 Java EditionのServer List

Pingに必要なHandshaking/Status状態だけを実装する。ログイン状態へ遷移しない。暗号化・圧縮・認証は不要。

9.1 フレーミング

- 各パケットは Length VarInt + Packet ID VarInt + Data で構成する。LengthはPacket IDとDataの合計バイト数。
- VarIntは最大5バイトとして読み、継続超過はProtocolError。負値や1 MiB超のpacket lengthは拒否する。
- 文字列はByte Length VarInt + UTF-8 bytes。宣言長と実読取長の不一致をエラーにする。
- Server Portはunsigned short、ネットワークバイトオーダー（big endian）。Ping payloadはsigned 64-bit big endian。

9.2 パケットシーケンス

順	方向	状態 / Packet ID	内容
1	Client → Server	Handshaking / 0x00	protocol version、server address、server port、next state=1(Status)。
2	Client → Server	Status / 0x00	Status Request。payloadなし。
3	Server → Client	Status / 0x00	Status Response。JSON string。
4	Client → Server	Status / 0x01	Ping Request。送信時刻由来のint64 nonce。
5	Server → Client	Status / 0x01	Pong Response。同じint64 payloadであることを検証。

9.3 Handshakeのprotocol version

Status取得は多くのサーバーでクライアントのプロトコル番号に依存せず応答するが、完全保証はない。MVPではビルド時定数として検証済みの現行値を保持し、protocolVersionを差し替え可能にする。将来は設定またはフォールバック候補を導入できるが、むやみに複数回接続して負荷を増やさない。

実装注意 バージョン表をアプリに大量同梱するより、Status Responseのversion.protocolを結果として保持する。MVPの目的はバージョン判定ではなく稼働確認である。

9.4 Status JSON

JSONパス	型	扱い
version.name	string	Version表示。空/欠落は不明。
version.protocol	number	内部保持し、将来の詳細表示/診断へ利用。
players.max	number	0以上を期待。上限の常識値は仮定しない。
players.online	number	0以上を期待。max超過でも即失敗せず警告候補。
players.sample	array	MVPでは非表示・無視。将来表示に備えdecode可能。
description	string/object/array	再帰的にtext/extraを抽出してMOTD化。未知要素は安全に無視。
favicon	data URI	MVPではdecodeしない。受信サイズ上限の対象。

9.5 MOTDの正規化

- descriptionが文字列ならそのまま採用する。オブジェクト/配列ならtextとextraを深さ制限付きで連結する。
- Minecraftの色・装飾情報はMVPでは捨て、可読なプレーンテキストを優先する。legacyの\$カラーコードも除去する。
- NULその他の制御文字を除去し、CRLFをLFへ統一。UI表示は最大4096 UTF-8 bytes相当へ制限する。
- 再帰深度は32、要素総数は1024等の上限を設け、悪意あるJSONによる過剰処理を防ぐ。

9.6 タイムアウトと上限

項目	既定	備考
全体	5秒	DNS、接続、送受信を含む。将来設定範囲1〜30秒。
Dial	3秒上限	全体期限より短い方を採用。
Read/Write	全体deadline	net.Conn.SetDeadlineで適用。
パケット	1 MiB	超過はPayloadTooLarge。
Status JSON	1 MiB	packet上限内。decoderは未知フィールド許容。
MOTD	4 KiB表示	元JSONは必要最小限のみ保持。

9.7 DNS SRV

MVPは入力されたhostとportへ直接接続する。portが省略されたときも25565を使い、_minecraft._tcp SRVは解決しない。将来SRV対応を追加する場合は、明示portを最優先し、SRVのpriority/weight規則、元のhostnameをHandshakeへ送るか実接続先を送るかを実サーバーで検証する。

9.8 MODサーバーの扱い

判定の意味 MODサーバーでも標準Status Responseを取得できればONLINEと判定する。ただし、この結果は『サーバー一覧への応答』を示すだけで、利用者のクライアントが必要MOD・ローダー・バージョンを満たし、実際にログインできることは保証しない。

サーバー側の状態	本ツールの挙動	注意
Forge / NeoForge / Fabric等が標準Statusに応答	通常どおりONLINE、Ping、Version、Players、MOTDを表示。	MOD一致確認はログイン段階で行われる場合がある。
既知のMOD拡張フィールドを含む	基本結果を優先し、modinfo / forgeData等を制限付きで解析。	ローダー名・MOD数は取得できた場合だけ補足表示。

サーバー側の状態	本ツールの挙動	注意
拡張フィールドがない	基本結果のみ表示し、MOD情報は『未検出』。	未検出をVanilla確定と解釈しない。Fabricや設定次第では一覧を公開しない。
MOD一覧が巨大・不正	基本JSONを安全に処理できなければERROR。『応答が大きすぎる/解釈できない』と表示。	サーバープロセス自体は稼働中の可能性がある。
独自プロキシ・MODがStatusを変更/無効化	有効なStatusを得られなければOFFLINEまたはERROR。	『停止中』と断定せず、Status非互換の可能性を案内。
起動途中・MOD読み込失敗	接続拒否、timeout、切断、またはメンテナンスMODとして観測。	1回の結果だけでクラッシュ原因や起動完了を判断しない。

9.8.1 MOD情報の表示規則

- MVPの必須表示項目は変えない。MOD情報は補助行『MOD: 検出 (Loader、N mods)』として、根拠がある場合だけ表示する。
- legacy Forge系のmodinfo、比較的新しいForge系のforgeDataなど既知形状をbest-effortで読む。フィールド名や形状はローダー/版で異なるため、解析失敗をサーバー全体のOFFLINEへ直結させない。
- MOD一覧が取得できても個別MOD名はMVP画面に列挙しない。ModCountとLoader推定だけをResultへ格納し、詳細一覧は将来の折りたたみ表示用とする。
- 拡張フィールドが存在しない場合はModded=falseではなくModInfoDetected=falseとする。『Vanilla』表示はサーバー種別を確実に識別できる追加根拠がない限り行わない。
- Status Response成功後に拡張部だけが不正な場合は警告付きONLINEを優先する。ただしJSON全体またはpacket境界が不正ならProtocol/Payload ERRORとする。
- 受信上限1
MiB、再帰深度、配列件数の制限はMOD情報にも適用し、巨大なFML/Forge互換情報によるメモリ消費を防ぐ。

9.8.2 データモデル追加

フィールド	型	意味
ModInfoDetected	bool	既知のMOD拡張情報を応答中に確認できたか。
ModLoader	string / optional	Forge、NeoForge等。応答から根拠を得た場合のみ。
ModCount	int / optional	公開されたMOD配列の件数。非公開・解析不能はnil。
ModInfoWarning	string / optional	拡張情報が切詰め、不正、上限超過等の補助診断。

10. ビルド・配布

10.1 開発環境

項目	要件
OS	Windows 10/11 x64を推奨。
Go	プロジェクトで固定した安定版。
C toolchain	Fyne desktopビルドに必要。MSYS2/MinGW-w64等をチーム内で統一。
Fyne CLI	アイコン・メタデータを埋め込むpackage工程で使用。バージョン固定。
ネットワーク	初回依存取得時のみ必要。リリースはgo.sum検証。

10.2 リリースビルド

1. go test ./... と静的解析を通す。

2. FyneのpackageコマンドでWindows EXEを生成し、アイコンとアプリメタデータを埋め込む。
3. -trimpathおよびリリース用ldflagsでversion/commit/buildDateを埋め込む。不要なコンソールを表示しないGUIサブシステムで生成する。
4. クリーンなWindows環境で、EXE単体から起動・Status取得・終了を確認する。
5. SHA-256を生成し、ファイル名mc-server-checker_vX.Y.Z_windows_amd64.exeで保管する。

10.3 単一EXEの定義と注意

配布条件 assets、アイコン、既定設定はgo:embed/Fyne resourceでEXEへ格納する。EXE横にDLL、JSON、HTML、設定ファイルを置かない。利用時にGo、Cコンパイラ、Fyne CLIは不要。

Windows SmartScreenは未署名の個人ビルドを警告する可能性がある。個人利用MVPではコード署名を必須にしないが、第三者配布へ進む場合はAuthenticode署名、配布元、SHA-256、変更履歴を整備する。UPXはウイルス対策ソフトの誤検知や署名運用を複雑にし得るため既定では使用しない。

10.4 設定保存

最後の接続先を保存する場合はFyne Preferencesを使用し、保存失敗でも主要機能を継続する。保存値はhost/portと任意のUI設定だけとし、認証情報は扱わない。『完全ポータブルで痕跡を残さない』モードは将来オプションとする。

11. テスト仕様

11.1 単体テスト

ID	対象	試験内容	期待結果
UT-01	VarInt	0, 1, 127, 128, 255, MaxInt32、5バイト超	正しいencode/decode、超過はエラー。
UT-02	Packet	分割読取、短い読取、負/巨大length、未知packet ID	境界を越えて読まず分類エラー。
UT-03	Target	hostname、IPv4、[IPv6]、空白、port 0/65536	正規化または検証エラー。
UT-04	JSON	完全、欠落、未知字段、型違い、不正JSON	許容項目と不正項目を正しく分類。
UT-05	MOTD	string、text/extra、配列、§色、深い再帰、制御文字	安全なプレーンテキストと上限。
UT-06	Error map	net.DNSError、timeout、refused、EOF、protocol	安定したKindと日本語文言。
UT-07	Service	成功、失敗、cancel、遅いfake checker	状態・期限・goroutine終了が正しい。
UT-08	MOD拡張	modinfo、forgeData、未知形状、巨大mods配列、拡張部不正	基本結果を維持し、検出/警告/上限を正しく扱う。

11.2 結合テスト

ID	ケース	期待結果
IT-01	ローカルfake serverが正規Status/Pongを返す	ONLINE、全フィールド、RTTを取得。
IT-02	接続後に応答しない	5秒以内にTimeout。UIは復帰。
IT-03	Status JSON後にPongしない	警告付きONLINE、Ping取得失敗。
IT-04	HTTP等の非Minecraftサービス	ONLINEにせずProtocol/Offline相当。
IT-05	1 MiB超の宣言/実データ	確保前または制限読取で拒否。
IT-06	確認中にウィンドウ終了	cancel、socket close、プロセス正常終了。
IT-07	連続100回確認	リーク、固まり、古い結果上書きなし。
IT-08	Forge/NeoForge/Fabricの許可済みテストサーバー	標準項目を取得。MOD情報の有無でONLINE判定を壊さない。

11.3 UI・受入テスト

- 初回起動でAddress空、Port 25565、状態「未確認」が表示される。
- 確認中もウィンドウ移動・再描画ができ、確認ボタンは二重実行を起こさない。
- ONLINE/OFFLINEが色なしでも文字で判別できる。
- 長いMOTD、日本語、絵文字、改行でレイアウトが崩れない。
- 100/125/150/200% DPI、ライト/ダークで文字切れや重なりがない。
- コピー内容が画面値と一致し、未取得値を誤って0として出さない。
- ネットワークなし、DNS失敗、閉じたportで理解可能なメッセージを出す。
- クリーンなWindows 10/11 x64でEXE以外を置かず起動する。

11.4 実サーバー確認

公開サーバーへの自動テストは負荷・規約・不安定性のため必須CIにしない。リリース前に所有または許可されたJavaサーバー（Vanilla系、Paper系、プロキシ経由の少なくとも2種類）で手動スモークテストを行い、取得値を公式クライアントのサーバー一覧と比較する。

12. 開発ステップ

段階	作業	完了条件
0. 骨格	Go module、CI、domain型、Checker interface、ビルド情報	go test ./... が空実装でも通る。
1. Codec	VarInt、string、packet framing、上限、deadline	境界値/異常系単体テスト通過。
2. Java Status	Handshake、Status JSON、Ping/Pong、MOTD	fake server結合テスト通過。
3. App層	入力検証、timeout、error mapping、requestID	UIなしのサービス試験通過。
4. GUI	入力、状態、結果、非同期更新、コピー	主要UI受入項目通過。
5. Packaging	アイコン、manifest、version、単一EXE	クリーンWindowsで単体起動。
6. Hardening	上限、cancel、リーク、DPI、長文MOTD	回帰/連続実行試験通過。
7. Release	README、SHA-256、手動スモーク、v1.0.0	受入基準を全件記録。

12.1 実装順の意図

最初にプロトコルとサービスをGUIなしで完成させる。ネットワーク不具合とUI不具合を分離でき、将来CLIや自動テストを追加しやすい。GUI着手時にはfake Checkerを注入し、オンライン/オフライン/遅延/不正応答を確実に再現する。

13. 将来拡張

段階	機能	設計上の準備	注意点
v1.1	監視モード	CheckService再利用、Tickerはapp層、停止context	最短間隔10秒、重複実行禁止。
v1.1	履歴	Resultを時系列モデル化、storage interface	件数上限、CSV/JSONエクスポート。
v1.2	JSON出力	domain.Resultにjson tag、CLI cmd追加	GUI EXEとCLI EXEを分けるか判断。
v1.2	SRV対応	Resolver interface、明示port優先	priority/weightと表示Target。
v2.0	Bedrock	protocol/bedrock実装、Edition enum	UDP Unconnected Ping、Javaと異なるtimeout。
v2.0	自動判定	Java/Bedrock Checkerのorchestrator	同時試行の負荷、曖昧な失敗表示。
将来	通知	状態変化イベント	OS通知、連続失敗抑制、個人設定。

13.1 Bedrock設計方針

JavaのTCP Checkerへ条件分岐を追加せず、Bedrock用UDP Checkerを別実装する。Resultは共通項目（状態、遅延、バージョン、プレイヤー、MOTD）を維持し、edition固有情報はDetails mapではなく型付き拡張へ格納する。自動判定はJava/Bedrockの成否と入力portを考慮し、両方失敗時に断定的なedition表示をしない。

13.2 監視・履歴方針

- 監視は固定間隔で、前回処理が終わるまで次回を開始しない。
- 履歴には確認時刻、対象、状態、Latency、Players、Version、ErrorKindを保存する。生のOSエラー全文は保存しない。
- 保存先、保持件数、削除、エクスポートを利用者が把握できる画面にする。
- 状態変化通知にはチャタリング防止（例: 2回連続失敗でOffline確定）を選択可能にする。

14. MVP受入基準

ID	合格条件	確認
AC-01	EXE単体をWindows 10/11 x64で起動できる。	☐
AC-02	AddressとPortを入力し、Enterまたは確認ボタンで要求を開始できる。	☐
AC-03	有効なJavaサーバーでONLINE、Ping、Version、Players、MOTDを表示できる。	☐
AC-04	閉じたport、DNS失敗、timeoutを固まらずOFFLINEとして案内できる。	☐
AC-05	非MinecraftサービスをMinecraft ONLINEと誤判定しない。	☐
AC-06	処理中もUIが応答し、二重実行と古い結果の上書きがない。	☐
AC-07	不正/巨大応答でクラッシュや過大メモリ確保を起こさない。	☐
AC-08	100～200% DPIと長い日本語MOTDで主要要素が切れない。	☐
AC-09	終了時に通信をキャンセルし、プロセスが残らない。	☐
AC-10	単体・結合テスト、実サーバースモーク、SHA-256生成をリリース記録に残す。	☐
AC-11	MODサーバーのStatus応答を通常どおり判定し、MOD情報非公開をVanilla確定と誤表示しない。	☐

リリース判定 AC-01～AC-11 をすべて満たし、既知問題が結果の誤判定・クラッシュ・配布不能に該当しない場合にv 1.0.0をリリース可能とする。

付録A. 既定値

設定	既定値	MVPで変更可能か
Port	25565	画面で変更可
全体timeout	5秒	不可（将来設定）
Packet上限	1 MiB	不可
MOTD表示上限	4 KiB相当	不可
Edition	Java	不可
監視間隔	なし	監視機能は将来
履歴保存	なし	将来

付録B. 参考情報

資料	用途	URL
Wails v2 Installation	WindowsではWebView2 Runtimeが必要。	参照
Wails Windows Guide	WebView2未導入時のdownload/embed/browser/error戦略。	参照
Fyne Packaging for Desktop	Windows EXEへのアイコン/メタデータ埋込とrelease packaging。	参照
Fyne Bundling Resources	Go/Fyneアプリの単一バイナリとリソース埋込。	参照
Fyne Repository	開発時のGo/C compiler前提とFyneの概要。	参照
Walk Repository	Windows GUI toolkit、manifest埋込、windowsgui build。	参照
PrismarineJS minecraft-data protocol	Java/Bedrockのプロトコルデータとバイトオーダーの参考。	参照
itzg/mc-monitor	一部Forge/FML2サーバーで巨大なMOD互換情報がStatus処理へ影響する実装上の注意。	参照
NeoForge mod files	MODの表示/互換性メタデータとサーバー一覧でのversion display testの参考。	参照
Fabric API mod-list proposal	Fabricでは一般的なStatus応答だけからMOD一覧を常に取得できるとは限らない点の参考。	参照

付録C. 変更管理

プロトコル、依存、既定timeout、パケット上限、エラー分類、受入基準を変更する場合は文書版を更新する。UIだけの変更でも、状態遷移または表示規則へ影響する場合は関連要件IDとテストを同時に改訂する。

版	日付	変更内容
1.1	2026-08-18	MODサーバーの判定、拡張フィールド、誤認防止、上限、テストを追加。PDF版を作成。
1.0	2026-08-18	初版。MVPのFyne採用、Java Status、単一EXE、拡張方針を定義。